Mathematical Foundations, Optimization Dynamics, and Memory Bounds of Low-Rank Adaptation (LoRA)

Chief Strategist J

June 13, 2026

Abstract

This technical brief presents the complete mathematical formulation, optimization mechanics, and memory bounds of Low-Rank Adaptation (LoRA), a prominent Parameter-Efficient Fine-Tuning (PEFT) method. We provide full derivations for the forward and backward passes, analyze the SVD motivation, dissect the gradient updates, and perform an analytical memory complexity analysis comparing full fine-tuning with LoRA. Detailed pseudocodes for execution and weight merging are provided.

Contents

Introduction to Parameter-Efficient Fine-Tuning (PEFT)

As Large Language Models (LLMs) scale to hundreds of billions of parameters, full parameter fine-tuning becomes increasingly intractable. During full fine-tuning, the optimizer states (e.g., in AdamW) scale linearly with the parameter count, requiring massive memory overhead. Additionally, storing separate checkpoints for different downstream tasks is prohibitively expensive.

Parameter-Efficient Fine-Tuning (PEFT) addresses these issues by freezing the pretrained base model parameters and training only a small set of task-specific adapter parameters. This approach:

1. **Reduces memory consumption** by orders of magnitude, allowing local training on consumer-grade hardware.
2. **Prevents catastrophic forgetting** by preserving the base model's generalized knowledge.
3. **Enables multi-task deployment** using a single frozen base model copy shared across multiple small adapter sets.

Among PEFT methods, **Low-Rank Adaptation (LoRA)** stands out by introducing no inference latency, as the trained adapter weights can be analytically merged back into the base weights prior to deployment.

Mathematical Foundations of Low-Rank Adaptation (LoRA)

Low-Rank Approximation Hypothesis

During fine-tuning, a weight matrix $W_0 \in \mathbb{R}^{d\times k}$ undergoes an update $\Delta W \in \mathbb{R}^{d\times k}$. The core hypothesis of LoRA is that the parameter update matrix ΔW has a low intrinsic rank $r \ll \min(d, k)$.

From Singular Value Decomposition (SVD), any rank-R matrix ΔW can be factored as:

$$\Delta W = U\Sigma V^\top = \sum_{i=1}^{R} \sigma_i u_i v_i^\top \tag{1}$$

where $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_R > 0$ are the singular values. The Eckart-Young-Mirsky theorem states that the best rank-r approximation ($r < R$) under the Frobenius norm is given by truncating the SVD:

$$\Delta W_r = U_r \Sigma_r V_r^\top = \sum_{i=1}^{r} \sigma_i u_i v_i^\top \tag{2}$$

Rather than performing full-rank training and computing the SVD post-hoc, LoRA directly parameterizes the update matrix ΔW as the product of two low-rank matrices:

$$\Delta W = \frac{\alpha}{r} BA \tag{3}$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$. The hyperparameter α is a constant scaling factor, and r is the adapter rank.

Forward Pass Activation

For a linear layer with input activation vector $x \in \mathbb{R}^k$, the forward pass computes the modified output activation vector $h \in \mathbb{R}^d$:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx \tag{4}$$

To analyze this computation graph, we introduce the intermediate low-rank bottleneck activation $z \in \mathbb{R}^r$:

$$z = Ax \tag{5}$$

The output h can then be written as:

$$h = W_0 x + \frac{\alpha}{r} Bz \tag{6}$$

Initialization Schemes

To ensure that the adapter does not change the model's behavior at the beginning of training, the parameters are initialized as:

$$A_{i,j} \sim \mathcal{N}\left(0, \sigma^2 = \frac{1}{r}\right) \tag{7}$$

$$B_{i,j} = 0 \tag{8}$$

Because B is initialized to zero, the weight update matrix starts at zero:

$$\Delta W\big|_{t=0} = \frac{\alpha}{r}(0)A = 0 \tag{9}$$

This yields:

$$h\big|_{t=0} = W_0 x + 0 = W_0 x \tag{10}$$

This initialization guarantees that training begins from the exact output state of the pretrained base model, preventing initialization disruption.

Scaling Factor α

The scaling factor $\frac{\alpha}{r}$ is crucial. When the rank r is varied, setting α to a constant value ensures that the scale of the adapter updates remains consistent. This removes the need to re-tune learning rates and other hyperparameters when changing the rank r.

Optimization Dynamics and Backpropagation

Under a scalar loss function $\mathcal{L}$, gradients are backpropagated through the network. The base weight matrix W_0 is frozen (requires_grad = False). Gradients are computed only for the adapter weights A and B.

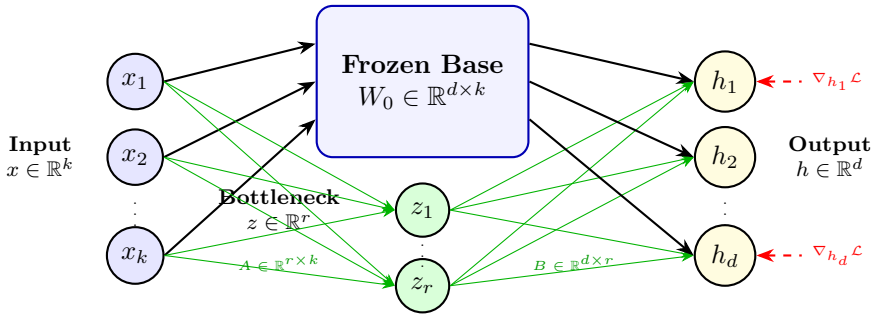


Figure 1: LoRA computational graph detailing parallel base and adapter paths and gradient backpropagation.

Analytical Derivation of Gradients

Let the incoming gradient with respect to the output layer be $\nabla_h \mathcal{L} = \frac{\partial \mathcal{L}}{\partial h} \in \mathbb{R}^{d\times 1}$. We derive the parameter gradients using the tensor chain rule.

Gradient with respect to Matrix B

Using the scalar chain rule for individual elements $B_{i,j}$:

$$\frac{\partial \mathcal{L}}{\partial B_{i,j}} = \sum_{m=1}^{d} \frac{\partial \mathcal{L}}{\partial h_m} \frac{\partial h_m}{\partial B_{i,j}} \tag{11}$$

Since $h_m = (W_0 x)_m + \frac{\alpha}{r}\sum_{l=1}^{r} B_{m,l} z_l$, the partial derivative is:

$$\frac{\partial h_m}{\partial B_{i,j}} = \begin{cases} \frac{\alpha}{r} z_j & \text{if } m = i \\ 0 & \text{otherwise} \end{cases} \tag{12}$$

Substituting this back yields:

$$\frac{\partial \mathcal{L}}{\partial B_{i,j}} = \frac{\alpha}{r} \frac{\partial \mathcal{L}}{\partial h_i} z_j \tag{13}$$

Expressing this in matrix notation as an outer product:

$$\nabla_B \mathcal{L} = \frac{\alpha}{r}(\nabla_h \mathcal{L}) z^\top = \frac{\alpha}{r}(\nabla_h \mathcal{L}) x^\top A^\top \quad \in \mathbb{R}^{d\times r} \tag{14}$$

Gradient with respect to Bottleneck Activation z

Before deriving the gradient for matrix A, we compute the gradient with respect to the bottleneck representations $z \in \mathbb{R}^r$:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \sum_{m=1}^{d} \frac{\partial \mathcal{L}}{\partial h_m} \frac{\partial h_m}{\partial z_j} \tag{15}$$

Since $h_m = (W_0 x)_m + \frac{\alpha}{r} \sum_{l=1}^{r} B_{m,l} z_l$, we have:

$$\frac{\partial h_m}{\partial z_j} = \frac{\alpha}{r} B_{m,j} \tag{16}$$

Substituting this back gives:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \frac{\alpha}{r} \sum_{m=1}^{d} B_{m,j} \frac{\partial \mathcal{L}}{\partial h_m} \tag{17}$$

In matrix notation, this corresponds to:

$$\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} \quad \in \mathbb{R}^{r \times 1} \tag{18}$$

Gradient with respect to Matrix A

Using the chain rule, we compute the gradient with respect to the elements of matrix A:

$$\frac{\partial \mathcal{L}}{\partial A_{j,k}} = \sum_{i=1}^{r} \frac{\partial \mathcal{L}}{\partial z_i} \frac{\partial z_i}{\partial A_{j,k}} \tag{19}$$

Since $z_i = \sum_{l=1}^{k} A_{i,l} x_l$, we have:

$$\frac{\partial z_i}{\partial A_{j,k}} = \begin{cases} x_k & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \tag{20}$$

This simplifies to:

$$\frac{\partial \mathcal{L}}{\partial A_{j,k}} = \frac{\partial \mathcal{L}}{\partial z_j} x_k \tag{21}$$

In matrix form:

$$\nabla_A \mathcal{L} = (\nabla_z \mathcal{L}) x^\top = \frac{\alpha}{r} B^\top (\nabla_h \mathcal{L}) x^\top \quad \in \mathbb{R}^{r \times k} \tag{22}$$

Backpropagation to Input Activation x

To pass the gradient back to the preceding layer, we compute the gradient with respect to the input activation vector $x \in \mathbb{R}^k$:

$$\nabla_x \mathcal{L} = W_0^\top \nabla_h \mathcal{L} + A^\top \nabla_z \mathcal{L} = W_0^\top \nabla_h \mathcal{L} + \frac{\alpha}{r} A^\top B^\top \nabla_h \mathcal{L} \quad \in \mathbb{R}^{k \times 1} \tag{23}$$

This shows that the backpropagated gradient is the sum of the gradient passing through the frozen base weights and the gradient passing through the low-rank adapter path.

Weight Merging at Inference

For inference, the forward equation $h = (W_0 + \frac{\alpha}{r} BA)x$ can be evaluated by merging the low-rank weights directly into the base weights. This eliminates the need to compute the parallel path, preventing any additional latency:

$$W_{\text{merged}} = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \tag{24}$$

If the adapter needs to be swapped out for a different task, the merge operation can be reversed:

$$W_0 = W_{\text{merged}} - \frac{\alpha}{r} BA \tag{25}$$

Memory Complexity Analysis

In this section, we analyze the memory footprint of the optimizer states. We compare full fine-tuning of a base weight matrix $W_0 \in \mathbb{R}^{d \times k}$ against low-rank training of A and B.

Let $N_{\text{base}} = d \cdot k$ be the number of base parameters, and $N_{\text{LoRA}} = r(d+k)$ be the number of LoRA parameters.

AdamW Optimizer Memory Model

AdamW tracks the following state elements for each trainable parameter:

1. **Parameter value copy** (often stored in FP32 master weights during mixed-precision training): 4 bytes.
2. **First moment vector m (momentum)**: 4 bytes.
3. **Second moment vector v (variance)**: 4 bytes.

This results in $M_{\text{opt}} = 12$ bytes of optimizer state memory overhead per trainable parameter.

Analytical Comparison

Metric	**Full Fine-Tuning**	**LoRA Fine-Tuning**
Trainable Parameters	$d \cdot k$	$r(d+k)$
Optimizer State Memory	$12 \cdot d \cdot k$ bytes	$12 \cdot r(d+k)$ bytes
Parameter Storage Overhead	High (Gigabytes)	Low (Megabytes)
Activation Memory	Baseline	Baseline + $\mathcal{O}(b \cdot s \cdot r)$
Gradient Memory	$2 \cdot d \cdot k$ bytes (FP16)	$2 \cdot r(d+k)$ bytes (FP16)

Table 1: Memory complexity comparison between Full Fine-Tuning and LoRA.

The memory saving ratio Φ is defined as:

$$\Phi = \frac{M_{\text{LoRA}}}{M_{\text{Full}}} = \frac{r(d+k)}{d \cdot k} = r\left(\frac{1}{d} + \frac{1}{k}\right) \tag{26}$$

Assuming a standard linear layer in a transformer block where $d = k = 4096$, and adapter rank $r = 8$:

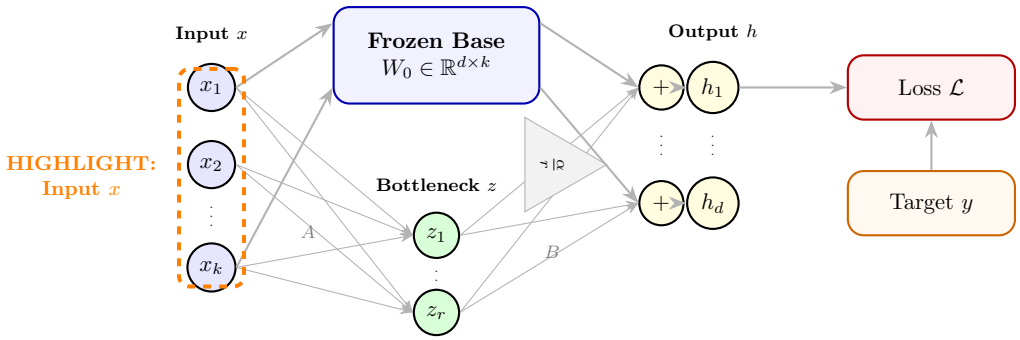
$$\Phi = 8\left(\frac{1}{4096} + \frac{1}{4096}\right) = \frac{16}{4096} = \frac{1}{256} \approx 0.39\% \tag{27}$$

This indicates a **256-fold reduction** (a decrease of 99.61%) in the memory required to store optimizer states for this weight matrix.

Detailed Operational Analysis of LoRA Variables

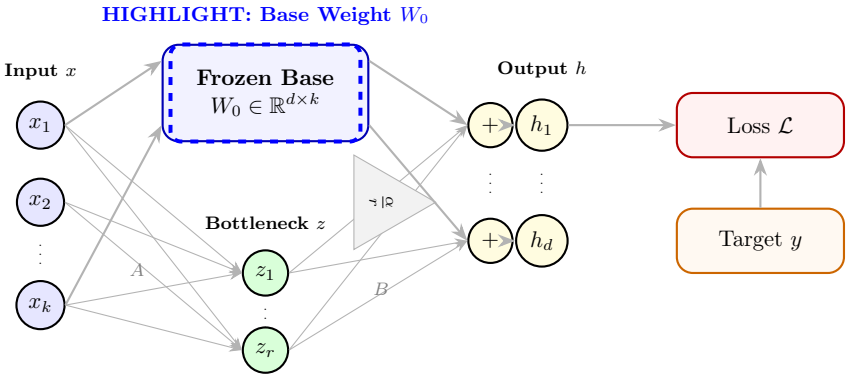
To understand the execution of LoRA at a granular level, we detail the mathematical nature of each variable, how its values are derived in reality, its child-level control dial mental model, and its physical mechanism at the individual neuron level. We provide a full neuron-level highlight diagram for each variable.

1. **Input Activation Vector (x)**:
    - **Formula**: x (output of previous layer)
    - **Neuron Highlight Diagram**:



    - **Mental Model**: Think of this vector as a list of positions for a row of thousands of physical dials. When the model processes a word, it adjusts these dials to specific numbers (e.g., dial 1 is at 4.2, dial 2 is at -1.5) to capture the meaning of the word. The model does not think in words; it thinks in these thousands of dial settings.
    - **Neuron-Level Physical Explanation**: A single neuron in the current layer receives x as a set of electrical signals from the neurons of the previous layer. Each element x_i represents the activation level (firing rate) of a previous neuron after its activation function (such as ReLU or GELU). The dendrites of the current neuron receive these signals.
    - **Mathematical Representation**: $x \in \mathbb{R}^{k \times 1}$
    - **Code Representation**: `x` (a PyTorch tensor of shape `(batch_size, seq_len, k)`)
    - **Relationships**: Fed into the base projection $W_0 x$ and compressed to $z = Ax$.

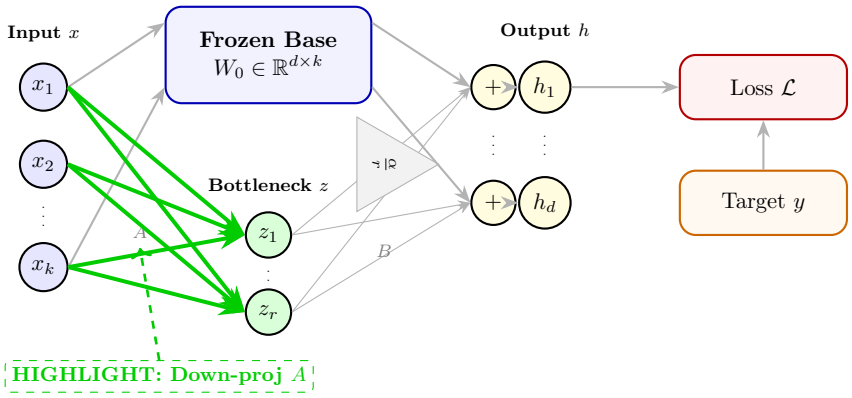
2. **Frozen Base Weight Matrix (W_0)**:
    - **Formula**: W_0
    - **Neuron Highlight Diagram**:



- **Mental Model**: This is a giant, permanent circuit board inside the control room. The wires are soldered in place and cannot be moved (frozen). They take the positions of the incoming dials (x), mix them together according to the permanent rules, and output a new set of general dial positions.
- **Neuron-Level Physical Explanation**: A single neuron's base connections are represented by a row of W_0. The neuron performs a dot product: it multiplies each incoming signal x_i by the strength of its corresponding synaptic connection $W_{0,ji}$ and sums them up: $y_j = \sum_i W_{0,ji} x_i$. During LoRA, these synaptic weights are frozen and cannot change, meaning their chemical/electrical conductance is locked.
- **Mathematical Representation**: $W_0 \in \mathbb{R}^{d \times k}$
- **Code Representation**: `W_0` (a PyTorch tensor with `W_0.requires_grad = False` to disable gradient computation)
- **Relationships**: Multiplied by the input to compute the baseline representation: $h_{\text{base}} = W_0 x$.

3. **Down-projection Adapter (A)**:

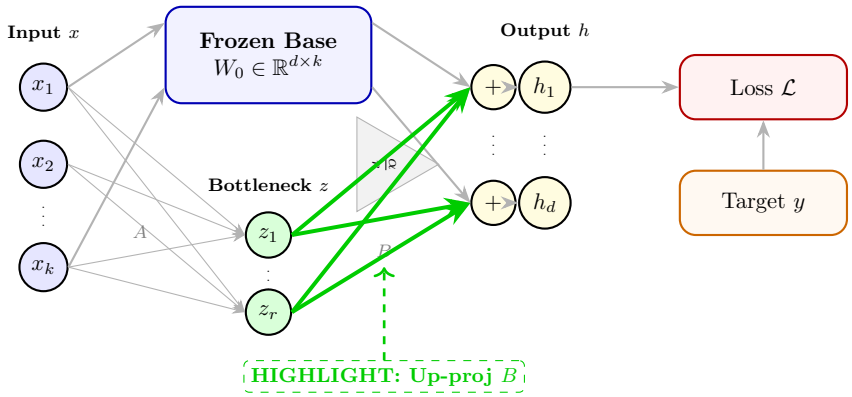
- **Formula**: A
- **Neuron Highlight Diagram**:



- **Mental Model**: This is a smaller circuit board that acts as a compression funnel. It looks at all the thousands of input dials and summarizes them into just a few dials (e.g., summarizing 4096 dials into just 8 dials). It only listens to the dial changes that are relevant to the new task we are teaching the model.
- **Neuron-Level Physical Explanation**: Inside the adapter path, a single neuron in the bottleneck layer corresponds to a row of matrix A. This neuron listens to all input signals x_i, multiplies them by its trainable weights A_{mi}, and sums them up: $z_m = \sum_i A_{mi} x_i$. This bottleneck neuron acts as a feature detector that extracts a specific combination of input signals.
- **Mathematical Representation**: $A \in \mathbb{R}^{r \times k}$
- **Code Representation**: `A` (initialized as `nn.Linear(k, r, bias=False)` with weight values drawn from a Gaussian distribution: `nn.init.kaiming_uniform_(A.weight, a=math.sqrt(5))`)
- **Relationships**: Compresses x into the bottleneck space via $z = Ax$.

4. **Up-projection Adapter (B)**:

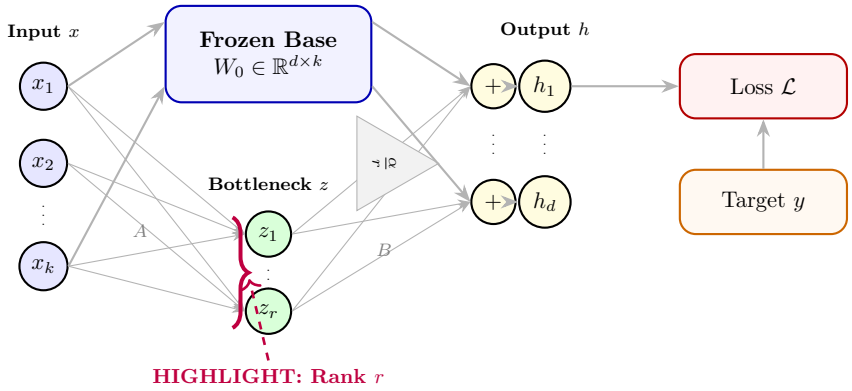
- **Formula**: B
- **Neuron Highlight Diagram**:



- **Mental Model**: This is another small circuit board that acts as an expansion nozzle. It takes the tiny summary from the funnel (our 8 dials) and expands it back into thousands of dials, showing how to nudge the final output dials to fit the new task.
- **Neuron-Level Physical Explanation**: A single output neuron receives contributions from the bottleneck neurons. The connection strength from bottleneck neuron m to output neuron j is the trainable weight B_{jm}. The output neuron sums the weighted signals from all bottleneck neurons: $u_j = \sum_m B_{jm} z_m$, scaling up the low-dimensional task features.
- **Mathematical Representation**: $B \in \mathbb{R}^{d\times r}$
- **Code Representation**: `B` (initialized as `nn.Linear(r,` d, bias=False) with weight values set to all zeros: `nn.init.zeros_(B.weight)`)
- **Relationships**: Expands the bottleneck activations via Bz, scaled by $\frac{\alpha}{r}$.

5. **Bottleneck Rank (r)**:

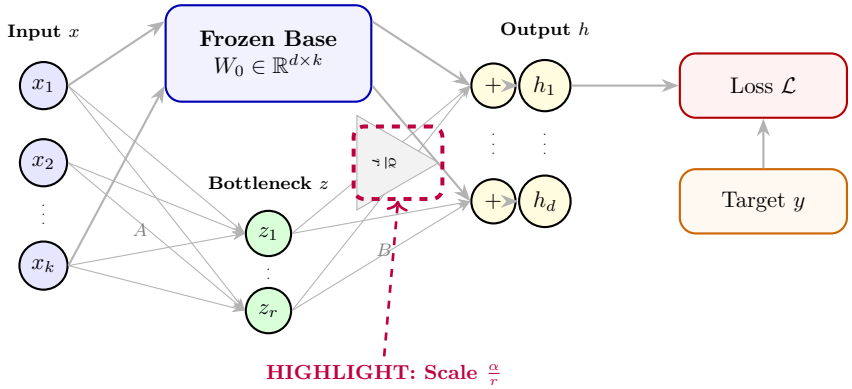
- **Formula**: r
- **Neuron Highlight Diagram**:



- **Mental Model**: This is the exact number of summary dials inside the bottleneck. If $r = 8$, it means we are forcing the model to summarize all its knowledge into just 8 dials. A smaller number makes the funnel tighter, forcing the model to only keep the most important summaries.
- **Neuron-Level Physical Explanation**: This is the exact number of neurons present in the bottleneck layer of the adapter. Selecting $r = 8$ means there are exactly 8 neurons in this intermediate layer. Each of these 8 neurons acts as a specialized task detector.
- **Mathematical Representation**: $r \in \mathbb{Z}^+$
- **Code Representation**: `r` (passed as an integer configuration parameter, e.g., `lora_config = LoraConfig(r=8)`)
- **Relationships**: Determines the inner dimension of matrices A and B, and acts as a scaling denominator in $\frac{\alpha}{r}$.

6. **Scaling Constant (α):**

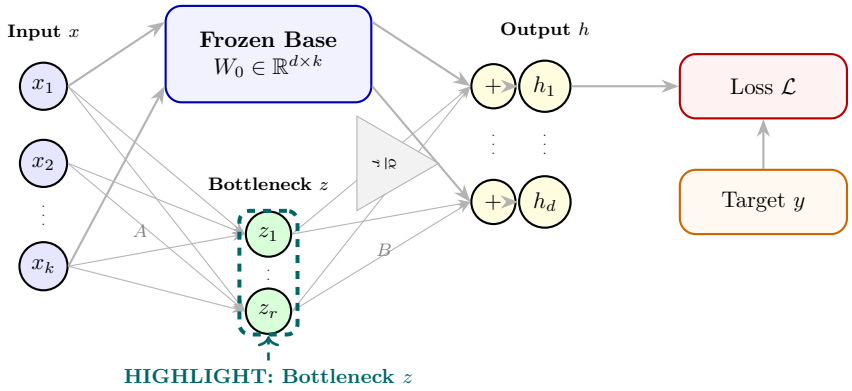
- **Formula**: α
- **Neuron Highlight Diagram**:



- **Mental Model**: This is a volume control knob. It decides how loudly the changes from the new adapters (A and B) should shout compared to the original, permanent circuit board (W_0). If we turn this knob up, the model listens more to its new training and less to its old pretrained knowledge.
- **Neuron-Level Physical Explanation**: At the neuron level, this acts as a synaptic gain control. The signal coming from the bottleneck via the adapter pathway is amplified or attenuated by a constant scale factor of $\frac{\alpha}{r}$ before it is added to the base neuron's output. This behaves like a biological neuromodulator that globally scales the synaptic sensitivity of the adapter path.
- **Mathematical Representation**: $\alpha \in \mathbb{R}^+$
- **Code Representation**: `lora_alpha` (passed as a configuration parameter, e.g., `lora_config = LoraConfig(lora_`
- **Relationships**: Multiplies the adapter pathway output, yielding $\frac{\alpha}{r}Bz$.

7. **Bottleneck Activation Vector (z):**

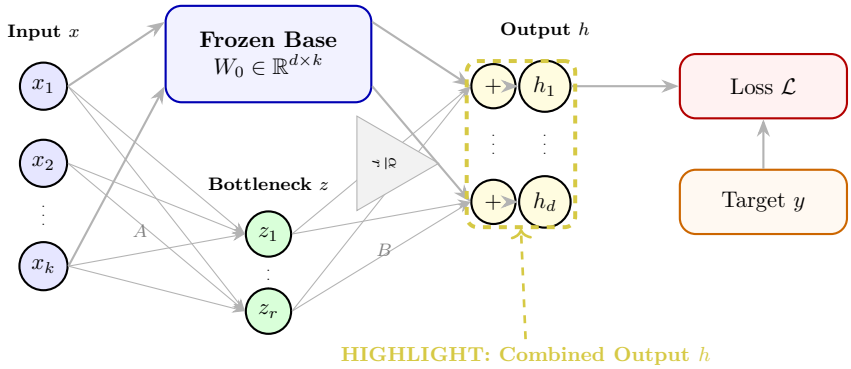
- **Formula**: $z = Ax$
- **Neuron Highlight Diagram**:



- **Mental Model**: These are the actual values of our few summary dials inside the bottleneck. It represents the compressed task-specific message as it flows from the compression funnel to the expansion nozzle.
- **Neuron-Level Physical Explanation**: Each element z_m is the electrical activation level (firing rate) of a single neuron in the bottleneck layer. It is calculated by that neuron summing its inputs: $z_m = \sum_i A_{mi}x_i$.
- **Mathematical Representation**: $z \in \mathbb{R}^{r\times 1}$

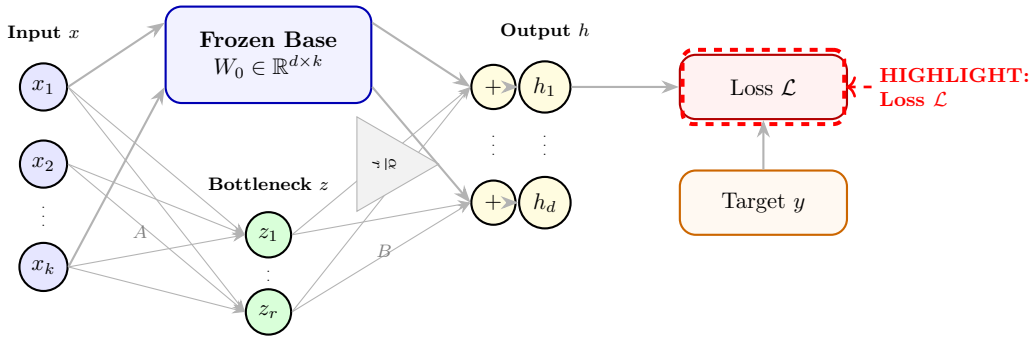
- **Code Representation**: `z = F.linear(x, A.weight)` (or `z = x @ A.weight.t()`)
- **Relationships**: Created by A, consumed by B.

8. **Combined Output Vector (h)**:
   - **Formula**: $h = W_0 x + \frac{\alpha}{r} Bz$
   - **Neuron Highlight Diagram**:



   - **Mental Model**: This is the final row of output dials. It is created by taking the general dial settings computed by the permanent circuit board (W_0) and nudging them slightly using the task-specific dial settings computed by our adapters.
   - **Neuron-Level Physical Explanation**: A single output neuron j sums the signals from the frozen base connection path and the scaled trainable adapter path: $h_j = \sum_i W_{0,ji} x_i + \frac{\alpha}{r} \sum_m B_{jm} z_m$. The neuron acts as a physical summing junction, integrating the general-purpose signal and the specialized task signal.
   - **Mathematical Representation**: $h \in \mathbb{R}^{d \times 1}$
   - **Code Representation**: `h = base_layer(x) + (alpha / r) * B(z)`
   - **Relationships**: Fed as input to subsequent layers of the network.

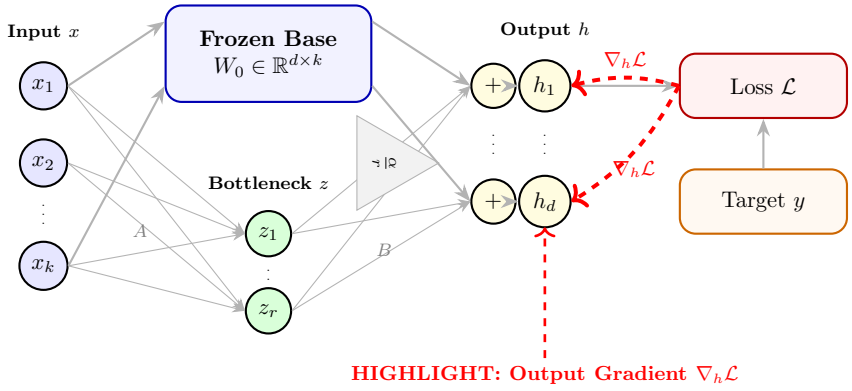
9. **Scalar Loss Value ($\mathcal{L}$)**:
   - **Formula**: $\mathcal{L}$ (e.g. Cross-Entropy loss)
   - **Neuron Highlight Diagram**:



   - **Mental Model**: This is a single scorecard number. It tells us how far off the final output dials are from what they should be. If the score is high, it means the model is making bad guesses. The goal of training is to adjust the new dials until the scorecard reads zero.

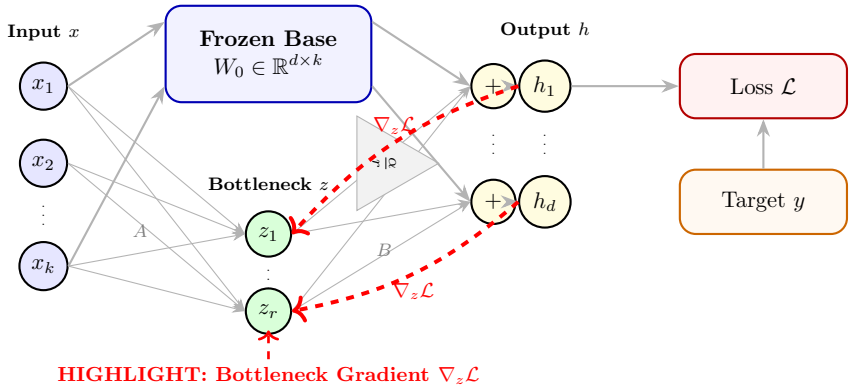
- **Neuron-Level Physical Explanation**: The loss $\mathcal{L}$ measures how far the firing rates of the output neurons in the final layer differ from the desired target firing rates. The target represents the correct next word, and the error signal is generated by comparing the target with the actual outputs.
- **Mathematical Representation**: $\mathcal{L} \in \mathbb{R}$
- **Code Representation**: `loss = F.cross_entropy(logits, targets)`
- **Relationships**: Traversed in reverse during backpropagation; all gradients are derivatives of $\mathcal{L}$.

10. **Output Layer Gradient ($\nabla_h \mathcal{L}$)**:
    - **Formula**: $\nabla_h \mathcal{L} = \frac{\partial \mathcal{L}}{\partial h}$
    - **Neuron Highlight Diagram**:



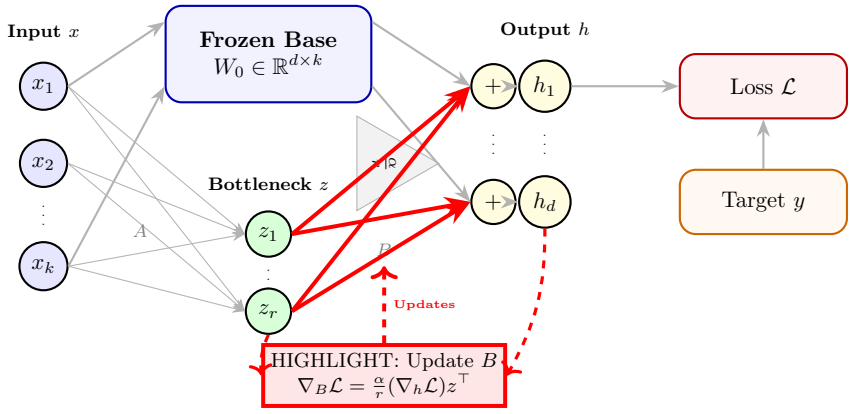
- **Mental Model**: This is a set of correction instructions for the final output dials. It says: "dial 1 needs to turn clockwise by 0.2, and dial 2 needs to turn counter-clockwise by 1.5" to lower the error score.
- **Neuron-Level Physical Explanation**: For a single output neuron j, the element $(\nabla_h \mathcal{L})_j = \frac{\partial \mathcal{L}}{\partial h_j}$ represents the error feedback signal. It tells the neuron whether it fired too strongly (positive gradient) or too weakly (negative gradient) and by how much it needs to adjust its total output to minimize the error.
- **Mathematical Representation**: $\nabla_h \mathcal{L} \in \mathbb{R}^{d\times 1}$
- **Code Representation**: automatically tracked by PyTorch's `loss.backward()`, represented as the grad tensor associated with output `h.grad`
- **Relationships**: Directly drives the gradient computation for B and z.

11. **Bottleneck Layer Gradient ($\nabla_z \mathcal{L}$)**:
    - **Formula**: $\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L}$
    - **Neuron Highlight Diagram**:



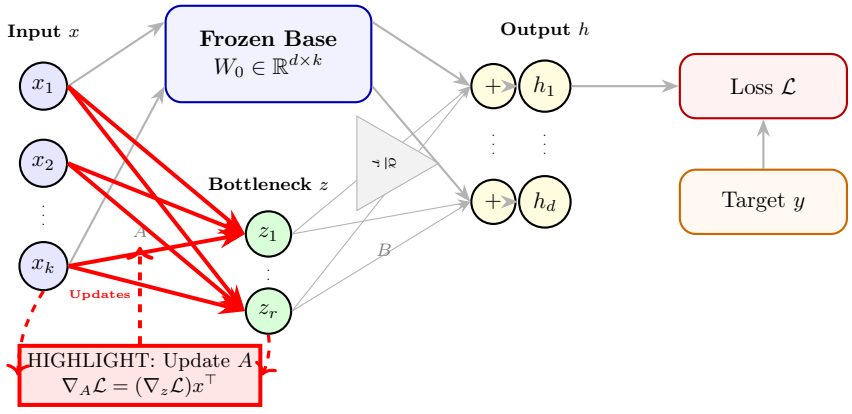
- **Mental Model**: This translates the correction instructions from the output room back down into the bottleneck. It tells the summary dials inside the funnel how they need to adjust to fix the output dials.
- **Neuron-Level Physical Explanation**: For a bottleneck neuron m, this is the backpropagated error signal. It is computed by summing the error feedback signals from all output neurons, weighted by their connections B_{jm} and scaled: $(\nabla_z \mathcal{L})_m = \frac{\alpha}{r} \sum_j B_{jm} (\nabla_h \mathcal{L})_j$. It tells the bottleneck neuron how its activation should have changed.
- **Mathematical Representation**: $\nabla_z \mathcal{L} \in \mathbb{R}^{r \times 1}$
- **Code Representation**: `grad_z = (alpha / r) * torch.matmul(grad_output, B.weight)`
- **Relationships**: Drives the gradient computation for A.

12. **Update Gradient for Matrix B ($\nabla_B \mathcal{L}$)**:
    - **Formula**: $\nabla_B \mathcal{L} = \frac{\alpha}{r} (\nabla_h \mathcal{L}) z^\top$
    - **Neuron Highlight Diagram**:



- **Mental Model**: This is the instruction sheet showing how to rewire the dials of the expansion nozzle (B) so that they do a better job of translating the summary back to the output dials.
- **Neuron-Level Physical Explanation**: For the synapse connecting bottleneck neuron m to output neuron j, the gradient is calculated as $(\nabla_B \mathcal{L})_{jm} = \frac{\alpha}{r} (\nabla_h \mathcal{L})_j z_m$. This follows Hebbian learning: the strength of the update is proportional to the product of the pre-synaptic activation (z_m) and the post-synaptic error signal ($(\nabla_h \mathcal{L})_j$).
- **Mathematical Representation**: $\nabla_B \mathcal{L} \in \mathbb{R}^{d \times r}$
- **Code Representation**: `B.weight.grad = (alpha / r) * torch.matmul(grad_output.t(), z)`
- **Relationships**: Used by the optimizer to update the values of B.

13. **Update Gradient for Matrix A ($\nabla_A \mathcal{L}$)**:
    - **Formula**: $\nabla_A \mathcal{L} = (\nabla_z \mathcal{L}) x^\top$
    - **Neuron Highlight Diagram**:



- **Mental Model**: This is the instruction sheet showing how to rewire the dials of the compression funnel (A) so that they do a better job of compressing the input dials into the summary.
- **Neuron-Level Physical Explanation**: For the synapse connecting input neuron i to bottleneck neuron m, the gradient is $(\nabla_A \mathcal{L})_{mi} = (\nabla_z \mathcal{L})_m x_i$. This also follows Hebbian learning: it multiplies the pre-synaptic input signal (x_i) by the post-synaptic bottleneck error signal ($(\nabla_z \mathcal{L})_m$) to determine how much the synapse should strengthen or weaken.
- **Mathematical Representation**: $\nabla_A \mathcal{L} \in \mathbb{R}^{r \times k}$
- **Code Representation**: `A.weight.grad = torch.matmul(grad_z.t(), x)`
- **Relationships**: Used by the optimizer to update the values of A.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of k = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{d \times k}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (k+d)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better,

but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{d\times k}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^d$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^k$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

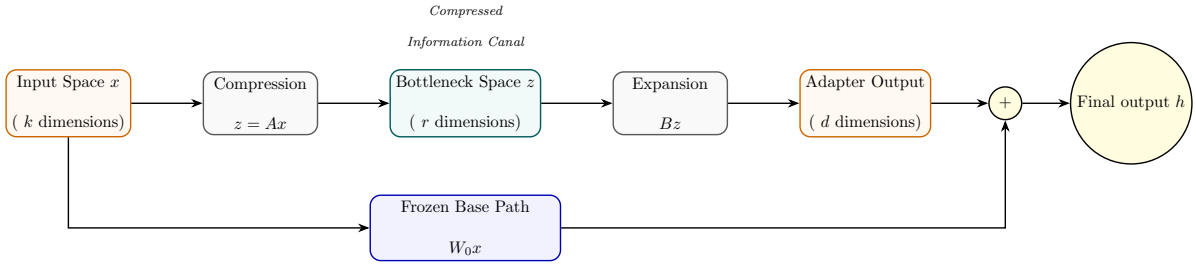


Figure 2: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{k\times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(k, d)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r\times 1}$.

- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of k = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{d\times k}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r\times(k+d)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{d \times k}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r} BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^d$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^k$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

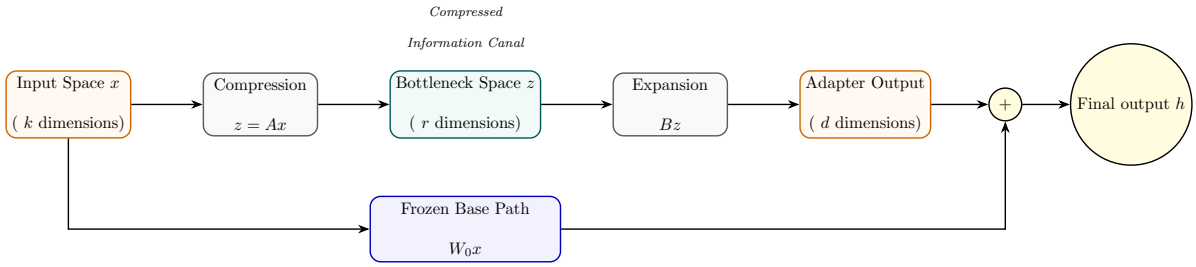


Figure 3: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{k \times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(k, d)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.

- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

In the context of Large Language Models, the embedding space has an extrinsic dimension of k = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{d\times k}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($mathbbR^{rtimes(k+d)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{d\times k}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of

dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^d$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^k$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

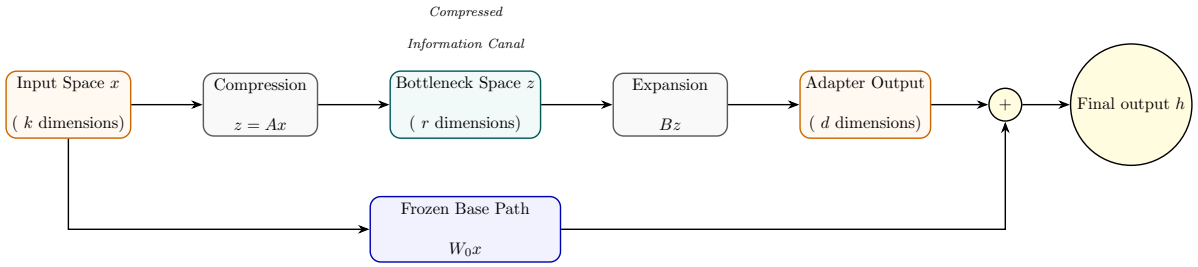


Figure 4: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{k \times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(k, d)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of k = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space

will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{d\times k}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($mathbbR^{rtimes(k+d)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{d\times k}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^d$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^k$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

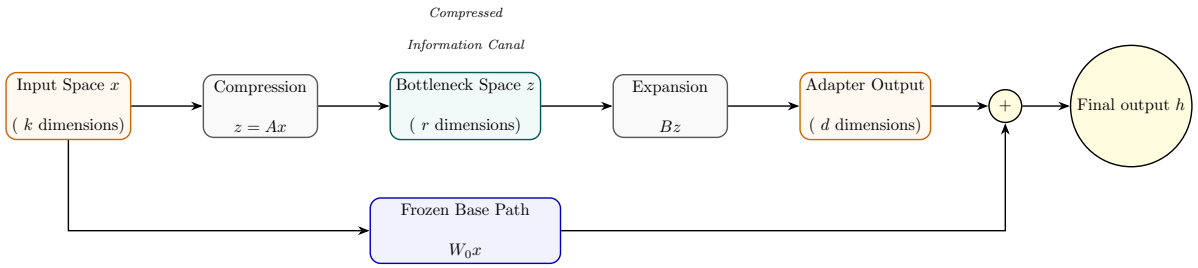


Figure 5: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{k \times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(k, d)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of k = 4096 (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{d\times k}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r\times(k+d)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{d\times k}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^d$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^k$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

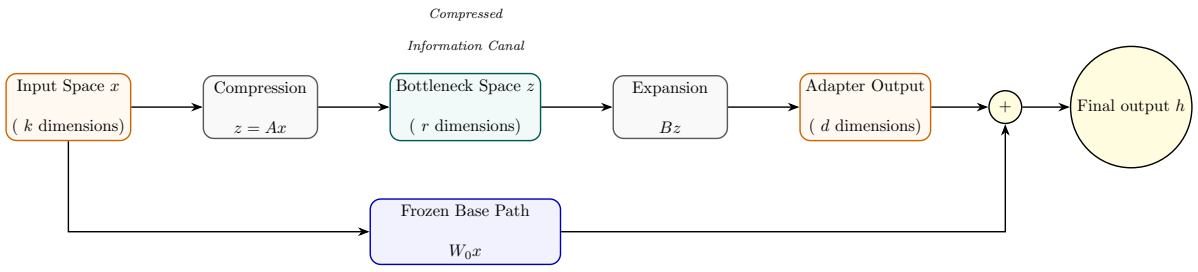


Figure 6: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{k \times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(k, d)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of $k = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{d \times k}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (k+d)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{d\times k}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.
- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^d$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^k$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

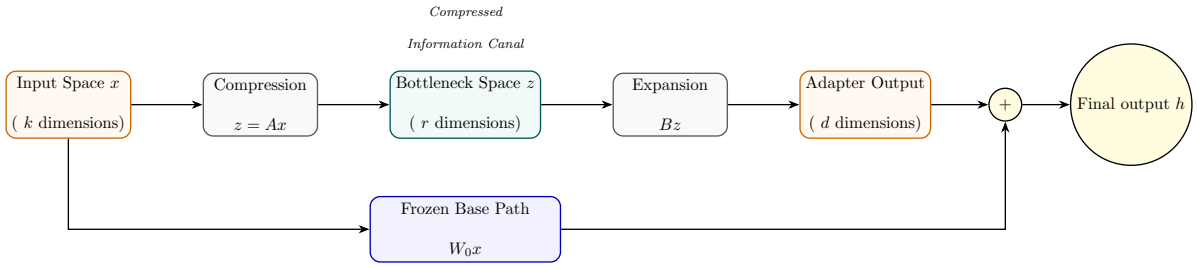


Figure 7: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{k\times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(k, d)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r\times 1}$.
- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of

Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument**: This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

  In the context of Large Language Models, the embedding space has an extrinsic dimension of $k = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold's tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument**: If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis**: While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam's Razor (Lex Parsimoniae)

- **Argument**: Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{d\times k}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

  By restricting the learning search to a compressed subspace ($\mathbb{R}^{r\times(k+d)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model's core foundational capabilities.

- **Counter-Argument**: By restricting the parameters to a low-rank product BA, we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam's Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis**: Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension**: A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{d\times k}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the

parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry**: The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r} BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A, rotates/scales them in this bottleneck space, and then projects them back to the output space via B. This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

  Geometrically, the column space of ΔW is spanned by the columns of B, which is an r-dimensional subspace of the output space $\mathbb{R}^d$. The row space of ΔW is spanned by the rows of A, which is an r-dimensional subspace of the input space $\mathbb{R}^k$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

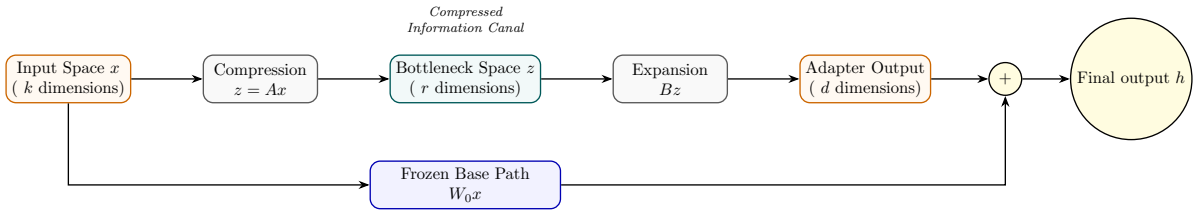


Figure 8: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression**: The activation vector $x \in \mathbb{R}^{k \times 1}$ is projected down through a narrow bottleneck of rank r. Since $r \ll \min(k, d)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow**: Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A, computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

LoRA CLRS-Style Algorithms

This section contains the CLRS-style mathematical algorithms for the forward pass, manual backward pass, parameter merging, and AdamW optimizer update step.

LoRA-Forward$(x, W_0, A, B, \alpha, r)$
  $h_{\text{base}} = W_0 x$
  $z = Ax$
  $z_{\text{out}} = Bz$
  $h = h_{\text{base}} + \frac{\alpha}{r} z_{\text{out}}$
  return h

Figure 9: LoRA-Forward algorithm: computes base output and scaled low-rank adapter output.

Analysis of Lora-Forward

- **What is achieved**: Computes the layer output h by parallelizing the computation of the frozen base weight path ($W_0 x$) and the trainable low-rank adapter path ($B(Ax)$), and scaling the adapter's contribution.

- **What internally changed**: Instantiates intermediate activation tensors $z = Ax \in \mathbb{R}^{r \times 1}$ and $z_{\text{out}} = Bz \in \mathbb{R}^{d \times 1}$ in memory. During training, these tensors are kept in GPU memory (VRAM) because their values are required to compute the gradients in the backward pass.
- **User-modifiable parameters & impact**:
  - **Rank** r (hyperparameter): Modifies the bottleneck dimension. A larger r (e.g., 32) captures more task-specific information but increases memory requirements and latency. A smaller r (e.g., 8) is highly memory-efficient but might underfit complex tasks.
  - **Scaling constant** α (hyperparameter): Adjusts the strength of the adapter's adjustments. Increasing α scales up the adapter's updates. Keep this constant when tuning r to stabilize learning.

Python Implementation (model.py) To implement the forward pass in PyTorch, we download a lightweight pre-trained model (like GPT-2) and wrap its linear projection layers in a custom LoraLinear wrapper module. The folder structure for this part of the project is:

```
my_lora_project/
+-- model.py
```

```python
import math
import torch
import torch.nn as nn
from transformers import AutoModelForCausalLM, AutoTokenizer

class LoraLinear(nn.Module):
    def __init__(self, base_layer: nn.Linear, r: int = 8, lora_alpha: float = 16.0):
        super().__init__()
        self.base_layer = base_layer
        self.r = r
        self.lora_alpha = lora_alpha
        self.scaling = lora_alpha / r

        in_features = base_layer.in_features
        out_features = base_layer.out_features

        # Keep base layer weights frozen (no gradients computed)
        self.base_layer.weight.requires_grad = False
        if self.base_layer.bias is not None:
            self.base_layer.bias.requires_grad = False

        # Define trainable low-rank adapters A and B
        self.lora_A = nn.Parameter(torch.zeros((r, in_features)))
        self.lora_B = nn.Parameter(torch.zeros((out_features, r)))

        # Standard LoRA initialization: Gaussian for A, Zeros for B
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
        nn.init.zeros_(self.lora_B)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Forward pass calculation: h = W_0 * x + (alpha / r) * B * A * x
        base_out = self.base_layer(x)
        adapter_out = (x @ self.lora_A.t()) @ self.lora_B.t()
        return base_out + self.scaling * adapter_out

if __name__ == "__main__":
    # Download a lightweight model (GPT-2) from Hugging Face
    model_name = "gpt2"
    print(f"Downloading base model: {model_name}")
    model = AutoModelForCausalLM.from_pretrained(model_name)
    tokenizer = AutoTokenizer.from_pretrained(model_name)

    # Instantiate a base linear layer and wrap it
    base_layer = nn.Linear(768, 768)
    lora_layer = LoraLinear(base_layer, r=8, lora_alpha=16.0)

    # Run a forward pass
```

```
    dummy_input = torch.randn(1, 10, 768)
    output = lora_layer(dummy_input)
    print("LoRA forward pass output shape:", output.shape)
```

Listing 1: PyTorch implementation of the LoRA Forward Pass

$$
\begin{aligned}
&\textsc{Lora-Backward}(x, A, B, \nabla_h \mathcal{L}, \alpha, r) \\
&\quad z = Ax \\
&\quad \nabla_B \mathcal{L} = \tfrac{\alpha}{r}(\nabla_h \mathcal{L}) z^\top \\
&\quad \nabla_z \mathcal{L} = \tfrac{\alpha}{r} B^\top \nabla_h \mathcal{L} \\
&\quad \nabla_A \mathcal{L} = (\nabla_z \mathcal{L}) x^\top \\
&\quad \nabla_{x,\text{adapters}} \mathcal{L} = A^\top \nabla_z \mathcal{L} \\
&\quad \textbf{return } \nabla_A \mathcal{L}, \nabla_B \mathcal{L}, \nabla_{x,\text{adapters}} \mathcal{L}
\end{aligned}
$$

Figure 10: Lora-Backward algorithm: derives gradients for adapters and backpropagated input gradient.

Analysis of Lora-Backward

- **What is achieved**: Calculates the exact gradients $\nabla_A \mathcal{L}$ and $\nabla_B \mathcal{L}$ for the adapter weights, and computes the backpropagated input gradient $\nabla_{x,\text{adapters}} \mathcal{L}$ to pass back to previous layers.
- **What internally changed**: Populates the gradient buffers `A.grad` and `B.grad` in VRAM. The frozen weights W_0 do not have gradient buffers, saving $k \times d \times 2$ or 4 bytes of memory.
- **User-modifiable parameters & impact**:
  - **Rank r and Scaling α**: Directly scale the gradients via the factor $\frac{\alpha}{r}$. Modifying them scales the gradient step size, affecting convergence stability.

Python Implementation (manual_grad.py) To verify the backward pass, we write a validation script that computes the manual gradients for A, B, and x based on the tensor chain rule, and compares them to PyTorch's autograd gradients. The folder structure for this verification is:

```
my_lora_project/
+-- manual_grad.py
```

```
import torch
import torch.nn as nn

def verify_gradients():
    # Set seed for reproducibility
    torch.manual_seed(42)

    # Dimensions: batch size=1, input dimensions k=4, output d=4, rank r=2
    k, d, r = 4, 4, 2
    alpha = 4.0
    scaling = alpha / r

    # Inputs and Targets
    x = torch.randn(1, k, requires_grad=True)
    target = torch.randn(1, d)

    # Base weight and adapters
    W0 = torch.randn(d, k)
    A = torch.randn(r, k, requires_grad=True)
    B = torch.randn(d, r, requires_grad=True)

    # --- PyTorch Autograd Path ---
    h_base = x @ W0.t()
    z = x @ A.t()
    h_adapter = z @ B.t()
    h = h_base + scaling * h_adapter

    # Mean Squared Error Loss
```

```
    loss = 0.5 * torch.sum((h - target) ** 2)
    loss.backward()

    # Extract Autograd Gradients
    grad_A_auto = A.grad.clone()
    grad_B_auto = B.grad.clone()
    grad_x_auto = x.grad.clone()

    # --- Manual Gradient Calculation ---
    # Gradient of loss w.r.t output h: dL/dh = h - target
    grad_h = h - target

    # grad_B = scaling * grad_h^T * z
    grad_B_manual = scaling * grad_h.t() @ z

    # grad_z = scaling * grad_h * B
    grad_z = scaling * grad_h @ B

    # grad_A = grad_z^T * x
    grad_A_manual = grad_z.t() @ x

    # grad_x = grad_h * W0 + grad_z * A
    grad_x_manual = grad_h @ W0 + grad_z @ A

    # --- Verification ---
    assert torch.allclose(grad_A_auto, grad_A_manual, atol=1e-6), "Gradients w.r.t A
     mismatch!"
    assert torch.allclose(grad_B_auto, grad_B_manual, atol=1e-6), "Gradients w.r.t B
     mismatch!"
    assert torch.allclose(grad_x_auto, grad_x_manual, atol=1e-6), "Gradients w.r.t x
     mismatch!"
    print("Manual gradients successfully verified against PyTorch Autograd!")

if __name__ == "__main__":
    verify_gradients()
```

Listing 2: PyTorch validation of manual LoRA gradients

LORA-MERGE$(W_0, A, B, \alpha, r, mode)$
$\Delta W = \frac{\alpha}{r} BA$
if $mode ==$ "merge"
$W = W_0 + \Delta W$
else if $mode ==$ "demerge"
$W = W_0 - \Delta W$
return W

Figure 11: LORA-MERGE algorithm: handles parameter folding and unfolding for zero-latency inference.

Analysis of Lora-Merge

- **What is achieved**: Merges the low-rank adapters directly into the base weights ($W = W_0 \pm \Delta W$) for zero-latency inference, or subtracts them to restore the base weights.
- **What internally changed**: In-place updates are applied to the base weight tensor W_0 in memory. Once merged, the adapter weights A and B can be discarded, and the model behaves as a single unified feedforward network, eliminating the need to compute separate low-rank paths during inference.
- **User-modifiable parameters & impact**:
  - **Mode**: User can choose `"merge"` to prepare the model for high-throughput deployment, or `"demerge"` to unpack the adapters and swap in a different set of adapter weights.

Python Implementation (merge.py) To deploy the model with zero inference latency, the adapter weights are folded directly into the base weights in-place. The folder structure for weight merging is:

```
my_lora_project/
+-- merge.py
```

```python
import torch
import torch.nn as nn

class MergeableLoraLinear(nn.Module):
    def __init__(self, base_layer: nn.Linear, r: int = 8, lora_alpha: float = 16.0):
        super().__init__()
        self.base_layer = base_layer
        self.r = r
        self.lora_alpha = lora_alpha
        self.scaling = lora_alpha / r

        self.lora_A = nn.Parameter(torch.randn(r, base_layer.in_features))
        self.lora_B = nn.Parameter(torch.randn(base_layer.out_features, r))
        self.merged = False

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        if self.merged:
            return self.base_layer(x)
        else:
            base_out = self.base_layer(x)
            adapter_out = (x @ self.lora_A.t()) @ self.lora_B.t()
            return base_out + self.scaling * adapter_out

    def merge_weights(self):
        if self.merged:
            return
        # W = W0 + (alpha / r) * B * A
        delta_W = self.scaling * (self.lora_B @ self.lora_A)
        self.base_layer.weight.data += delta_W
        self.merged = True
        print("Adapter weights merged into base weights.")

    def demerge_weights(self):
        if not self.merged:
            return
        # W0 = W - (alpha / r) * B * A
```

```
        delta_W = self.scaling * (self.lora_B @ self.lora_A)
        self.base_layer.weight.data -= delta_W
        self.merged = False
        print("Adapter weights demeged successfully.")

if __name__ == "__main__":
    # Instantiate base linear layer and wrap it
    base_linear = nn.Linear(512, 512)
    model = MergeableLoraLinear(base_linear, r=8, lora_alpha=16.0)

    dummy_input = torch.randn(1, 10, 512)

    # 1. Output before merging
    output_before = model(dummy_input)

    # 2. Merge weights
    model.merge_weights()
    output_after = model(dummy_input)

    # 3. Verify identical outputs
    assert torch.allclose(output_before, output_after, atol=1e-5), "Outputs do not
    match!"
    print("Inference verification passed. Merged output is identical.")

    # 4. Save state dictionary for latency-free deployment
    torch.save(base_linear.state_dict(), "merged_linear.pt")
    print("Saved merged state dict to 'merged_linear.pt'.")
```

Listing 3: Merging and demerging LoRA weights in-place

LoRA-AdamW-Update$(\theta, g, m, v, \eta, \lambda, \beta_1, \beta_2, \epsilon, t)$
for each parameter matrix $P \in \theta$ **do**
$P = P - \eta\lambda P$
$m_P = \beta_1 m_P + (1-\beta_1) g_P$
$v_P = \beta_2 v_P + (1-\beta_2) g_P^2$
$\hat{m}_P = m_P / (1-\beta_1^t)$
$\hat{v}_P = v_P / (1-\beta_2^t)$
$P = P - \eta \frac{\hat{m}_P}{\sqrt{\hat{v}_P}+\epsilon}$
return θ, m, v

Figure 12: LoRA-AdamW-Update algorithm: updates parameters with decoupled weight decay and momentum/variance tracking.

Analysis of Lora-AdamW-Update

- **What is achieved**: Applies decoupled weight decay to the adapter parameters and updates their values using momentum and variance of gradients.
- **What internally changed**: Trainable parameters A and B are updated. Additionally, the first moment vectors m_A, m_B and second moment vectors v_A, v_B are updated in GPU memory. Because W_0 is frozen, no optimizer states are created or updated for W_0, resulting in huge VRAM savings.
- **User-modifiable parameters & impact**:
  - **Learning Rate** η: Controls the step size. High values speed up training but risk divergence; low values ensure stability but slow down convergence.
  - **Weight Decay** λ: Regularizes the weights to prevent overfitting.
  - **Momentum coefficients** β_1, β_2: Control the memory of past gradients.

Python Implementation (train.py) To train the model, we write a complete end-to-end script that loads a lightweight pre-trained causal model, freezes its original layers, registers the trainable adapter weights, and runs a forward-backward optimization step using AdamW. The folder structure for this training file is:

```
my_lora_project/
+-- train.py
```

```python
import torch
import torch.nn as nn
from torch.optim import AdamW
from transformers import AutoModelForCausalLM

def train_lora_model():
    # Download a lightweight base model from Hugging Face
    model_name = "distilgpt2"
    print(f"Loading pre-trained base model: {model_name}")
    model = AutoModelForCausalLM.from_pretrained(model_name)

    # Freeze all original parameters in the base model
    for param in model.parameters():
        param.requires_grad = False

    # Inject adapters to the lm_head projection layer
    base_layer = model.lm_head
    r, alpha = 8, 16.0
    scaling = alpha / r

    # Define and initialize trainable adapter parameters A and B
    lora_A = nn.Parameter(torch.randn(r, base_layer.in_features) * 0.02)
    lora_B = nn.Parameter(torch.zeros(base_layer.out_features, r))

    # Register adapters as parameters of the model
    model.register_parameter("lora_A", lora_A)
    model.register_parameter("lora_B", lora_B)

    # Override the lm_head forward pass to compute base + adapter paths
    def custom_lm_head_forward(x):
        base_out = base_layer(x)
        adapter_out = (x @ lora_A.t()) @ lora_B.t()
        return base_out + scaling * adapter_out

    model.lm_head.forward = custom_lm_head_forward

    # Collect parameters that require gradients (only our custom adapters)
    trainable_params = [p for p in model.parameters() if p.requires_grad]
    print(f"Number of trainable parameters: {sum(p.numel() for p in trainable_params
    )}")

    # Setup AdamW optimizer specifically for the adapter parameters
    optimizer = AdamW(trainable_params, lr=1e-4, weight_decay=0.01)

    # Create dummy input tokens (Batch size 2, Sequence length 16)
    dummy_input = torch.randint(0, 50257, (2, 16))
    labels = dummy_input.clone()

    # Set model to training mode
    model.train()
    optimizer.zero_grad()

    # Forward pass through model (including modified lm_head)
    outputs = model(dummy_input, labels=labels)
    loss = outputs.loss

    # Backward pass to compute gradients for trainable adapters
    loss.backward()

    # Optimizer step applies the AdamW updates to parameters
    optimizer.step()
    print(f"AdamW update step completed. Current loss: {loss.item():.4f}")

if __name__ == "__main__":
```

```
train_lora_model()
```

Listing 4: Complete LoRA training and optimization loop using AdamW